

Week 10 PyTorch Portfolio Project Synthesis Workshop

NMTAFE - Cert IV Data Science and AI

Scaffolded support for your foundation skills portfolio; Applying PyTorch, Bash, and workflow knowledge.

Learning Objectives

- Apply core PyTorch fundamentals in a mini-project using tensors and autograd.
- Integrate Bash and Python scripting to automate ML workflows.
- Document, test, and reflect on code using industry standards.
- Prepare evidence and submissions for portfolio assessment.

Workshop Agenda

- Portfolio project overview and assessment alignment.
- Revisiting PyTorch tensors and autograd basics.
- Integrating Bash scripting for automation tasks.
- In-class coding activities and peer review.
- Documentation and assessment workshop.
- Q and A plus troubleshooting.

Industry Context and Application

- ML engineers use cross-language scripting to automate workflows.
- Cloud platforms and GPU compute require good automation habits.
- Structured documentation, code testing, and reproducibility are industry essentials.
- Secure scripting and proper code versioning support team-based development.

Portfolio Project Overview

- Your portfolio is evidence for TAFE and industry skills.
- Demonstrate PyTorch foundations; tensors, computation graphs, autograd, and basic neural layers.
- Use Bash or Python scripts to automate part of your data or training workflow.
- Document your process, decisions, and results for review.

Recap; Foundation Skills From Previous Weeks

- Week 5-6; PyTorch basics, tensor operations, autograd, model fundamentals.
- Week 2-4; Bash scripting, version control, pseudocode, automation methods.
- Week 7-9; Cloud concepts, data storage, secrets management.
- Bring these together into a cohesive mini-project.

Practical Activity; PyTorch Tensor and Autograd Workflow

- Task; Create a mini-project demonstrating use of tensor operations and autograd.
- Example; Linear regression with gradient computation and loss analysis.
- Goal; Code is reproducible, readable, and connected to a real data example.

```
import torch
# Example of linear regression with autograd
x = torch.randn(100, 1)
y = 2 * x + 1 + 0.1 * torch.randn(100, 1)
w = torch.randn(1, requires_grad=True)
b = torch.randn(1, requires_grad=True)
optimizer = torch.optim.SGD([w, b], lr=0.1)
for epoch in range(100):
    y_pred = x * w + b
    loss = ((y - y_pred) ** 2).mean()
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()
print(f"Trained weight: {w.item()} bias: {b.item()}")
```

Practical Activity; Automating with Bash and Python Scripting

- Use Bash or shell commands to automate steps; examples; data download, virtual environment setup, or start ML jobs.
- Example; Shell script to set up project directories and fetch a dataset.

```
#!/bin/bash  
mkdir -p ~/ml_project/data  
wget -O ~/ml_project/data/data.csv http://example.com/data.csv  
python preprocess.py ~/ml_project/data/data.csv
```

- Schedule your script with `cron` or test manual execution.

Documentation and Code Standards

- Use clear docstrings and comments; explain code purpose and function.
- Store experiment parameters in config files or environmental variables.
- Maintain a `README.md` for your project; describe setup, usage, dependencies, and experiment notes.
- Follow naming and versioning conventions; example; `model_v1.pt`.

Reflection and Code Testing

- Test scripts in clean environments to confirm all dependencies are listed and instructions are reproducible.
- Common pitfalls; hardcoded paths, missing package installs, lack of inline comments.
- Ask a peer to follow your `README.md` or setup steps; fix unclear instructions.

Preparing Your Portfolio Submission

- Structure; Jupyter notebooks (.ipynb), scripts (.py/.sh), documentation (.md), and evidence screenshots.
- Checklist; Tensors and autograd code, at least one Bash/Python automation script, documentation, output snapshots.
- Demonstrate industry practices; version control usage, clear variable names, workflow screenshots or logs.

Peer Review Activity

- Partner with another student to review each other's portfolio draft.
- Use the following checklist:
 - Can you run their scripts or notebooks without error?
 - Are all major steps and scripts documented?
 - Is the automation practical and reusable?

Troubleshooting and Support

- Common issues; package versions, missing dependencies, script permissions, file paths.
- Use `pip freeze` to document your environment; include in submission.
- Ask for help in class, by email, or book an appointment for support.

Summary and Next Steps

- Consolidate foundational skills across Python, Bash, and ML workflow management.
- Prepare and refine assessment evidence; seek feedback before submission.
- Next weeks; focus turns to ML experiment tracking, documentation, and moving workloads to cloud ML infrastructure.

